FedPEFT-IMU: Federated Parameter-Efficient Adaptation of Pretrained IMU Encoders for Human Activity Recognition under Device and Data Heterogeneity

First Author, *Student Member, IEEE*, Second Author, *Member, IEEE*

***Abstract*—Abstract to be written once the results section is final.**

***Index Terms*—Federated learning, human activity recognition, inertial sensors, low-rank adaptation, parameter-efficient fine-tuning, foundation models, personalization, heterogeneity.**

I. Introduction

[Introduction: to be written.]

II. Related Work

[Related work: to be written.]

III. Proposed Method: FedPEFT-IMU

This section formalizes the learning problem (Section III-A), presents the architecture and the role assigned to every parameter (Sections III-B and III-C), derives the client objective (Section III-D) and the server aggregation rules together with their exactness guarantees (Sections III-E and III-F), explains how label skew is handled (Section III-G), and describes inference for enrolled and new users (Section III-H). The complete procedure is summarized in Algorithm 1, and its cost is analyzed in Section III-K. Table I lists the main notation.

A. Problem Setting and Design Goals

We consider K users, each holding one device with an inertial measurement unit (IMU), e.g., a smartphone. The device records multi-axis signals (acceleration and, where available, angular velocity and magnetic field), segmented into windows $\mathbf{x} \in \mathbb{R}^{C\times L}$ (on HARBox $C = 9$ axes, 2 s at 50 Hz, $L = 100$) with activity labels $y \in \{1, \dots, C_y\}$; for readability we write C for the number of activity classes from here on. This gives a chronologically ordered dataset $\mathcal{D}_k = \{(\mathbf{x}_{k,i}, y_{k,i})\}_{i=1}^{|\mathcal{D}_k|}$. Raw windows never leave the device. A server orchestrates training over rounds $t = 0, \dots, T-1$ in which a random subset $\mathcal{S}_t$ of $\lceil \rho K \rfloor$ clients participates.

The setting combines four kinds of heterogeneity that, together, distinguish federated human activity recognition (HAR) from the image and text benchmarks on which most federated parameter-efficient fine-tuning (PEFT) methods were developed:

Manuscript draft. Code, user partitions and all generated statistics are released with the paper.

TABLE I
Main notation.

Symbol	Meaning
K, $\mathcal{S}_t$, ρ	number of clients, clients selected in round t, participation rate
$\mathcal{D}_k^{\mathrm{tr}}$, $\mathcal{D}_k^{\mathrm{te}}$	chronological training / test part of client k
n_k, $n_{k,c}$	training windows of client k (in total / of class c)
C, $\boldsymbol{\pi}_k$, $\boldsymbol{\pi}$	classes, label prior of client k, global label prior
Π_E, a_k	fixed preprocessor of encoder E, local input adapter
$\bar{W}_\ell$, $\mathcal{L}$	frozen base weight of adapted layer ℓ, set of adapted layers
B_ℓ, A_ℓ, r_k, s	LoRA factors, LoRA rank of client k, scaling $s = \alpha/r$
ψ_k	local normalization parameters and statistics
g_θ, h_k	shared projection head, personal classifier
$\mathbf{z}$, $\mathbf{e}$, $\bar{\mathbf{e}}$	encoder feature, embedding ($D = 64$), $\mathbf{e}/\|\mathbf{e}\|_2$
$\mathbf{p}_{k,c}$, $\mathbf{P}_c$	local and global prototype of class c
$\tilde{p}_k$	aggregation weight $n_k / \sum_{j\in\mathcal{S}_t} n_j$
$B_{[r]}$, $A_{[r]}$	first r columns of B / first r rows of A
$\Delta_{k,\ell}$	change of client k's LoRA product in one round
N_k, Γ	$C \times C$ transition counts of client k, HMM transition matrix

(H1) *Label skew*: $\boldsymbol{\pi}_k \neq \boldsymbol{\pi}$; users perform different activities, and different amounts of them.

(H2) *Feature shift*: $p_k(\mathbf{x} \mid y)$ differs across users because of device model, grip and orientation, body size and movement style; on HARBox this is the dominant heterogeneity (Section IV-D).

(H3) *System heterogeneity*: devices differ in memory and compute; we model this by a per-device budget expressed as a LoRA rank $r_k \in \mathcal{R} = \{2, 4, 8\}$.

(H4) *Temporal dependence*: activities last longer than a window, so consecutive windows are correlated; with overlapping windows they even share samples, which an evaluation protocol must respect (Section IV-C).

A pretrained IMU encoder E with weights Ω_0, obtained from data outside the federation, is available to every participant. The design goals are (G1) high macro-F1 for *enrolled* users on their *future* data; (G2) useful predictions for *new* users with zero or a few labelled windows; (G3) low uplink traffic; (G4) training within each device's budget r_k; and (G5) fairness, i.e., a high F1 for the worst-served users.

We cast the problem as personalized federated learning with shared parameters Θ and client-specific parameters ϕ_k:

$$\min_{\Theta} \sum_{k=1}^{K} p_k \min_{\phi_k} F_k(\Theta, \phi_k), \qquad p_k = \frac{n_k}{\sum_j n_j}, \tag{1}$$

TABLE II
COMPONENTS OF THE FEDPEFT-IMU CLIENT MODEL AND THEIR ROLES.

#	Component	Role	Purpose
①	Preprocessor Π_E	fixed	match the encoder's rate, length and units
②	Input adapter a_k	local	device axes / channel mapping
③	Base weights $\bar{W}_\ell$	frozen	generic motion representation
	LoRA $B_\ell A_\ell$ (rank r_k)	federated	population-level adaptation
	Norms ψ_k	local	per-user feature statistics (H2)
④	Shared head g_θ	federated	common 64-d embedding space
⑤	Personal head h_k	local	per-user decision rule (H1)
⑥	Prototypes $\mathbf{p}_{k,c}$, $n_{k,c}$	statistics	cross-user alignment, zero-shot
⑦	Decoder (prior, HMM)	statistics	calibration, temporal context (H4)

where F_k is the local objective defined in Section III-D and $n_k = |\mathcal{D}_k^{\mathrm{tr}}|$.

B. Architecture Overview and Parameter Roles

Figure 1 shows the complete system. Its central design decision is to give every tensor of the client model exactly one of four *roles*, chosen according to what the tensor is expected to capture (Table II):

- *Frozen*: the pretrained weights Ω_0. They encode generic motion features learned from large external corpora and are never transmitted after deployment.
- *Federated* (Θ): the LoRA factors $\{A_\ell, B_\ell\}_{\ell\in\mathcal{L}}$ and the projection head g_θ. They adapt the representation to the target population and benefit from all users' data.
- *Local* ($\phi_k = \{a_k, \psi_k, h_k\}$): the input adapter, the normalization layers and the classifier. Following the evidence that feature shift is largely captured by normalization statistics [1] and label skew by the classifier layer [2], [3], these parts absorb (H1)–(H2) and are never uploaded, which also reduces what a server can infer about an individual.
- *Statistics*: per-class prototypes $\mathbf{p}_{k,c}$ and counts $n_{k,c}$, sent every round, and transition counts N_k, sent once. They are small ($C \times D$, C and C^2 numbers) and align the users' embedding spaces (Section III-F).

Because each method studied in this paper is defined solely by which roles are trained, uploaded or kept local (Table VIII), all methods share one implementation, and differences in the results can be attributed to the role assignment rather than to engineering details.

C. Client Model

1) Encoder-agnostic preprocessing ①: Windows are stored once in a canonical format (3×300, 30 Hz, units of g). For each encoder E, a fixed operator Π_E maps them to the format used during pretraining: anti-aliased linear resampling to the encoder rate f_E, a centered crop to the encoder length L_E (e.g., 6 s for LIMU-BERT), and conversion to $\mathrm{m\,s^{-2}}$ where required. Π_E has no parameters and is identical on all devices.

2) Input adapter ②: A 1×1 convolution $a_k : \mathbb{R}^{3\times L_E} \to \mathbb{R}^{c_E \times L_E}$ maps the device channels to the c_E input channels of the encoder. It is initialized to the identity on the accelerometer channels and to zero elsewhere (e.g., on the gyroscope channels expected by LIMU-BERT), so training starts exactly from the pretrained model. With $3c_E + c_E$ parameters, it can learn device-specific axis permutations, sign flips and gains, and it is therefore kept local.

3) Frozen encoder with low-rank adapters ③: For every adapted layer $\ell \in \mathcal{L}$ with frozen weight $\bar{W}_\ell \in \mathbb{R}^{d_\ell^{\mathrm{out}} \times d_\ell^{\mathrm{in}}}$, LoRA [4] learns $B_\ell \in \mathbb{R}^{d_\ell^{\mathrm{out}} \times r}$ and $A_\ell \in \mathbb{R}^{r \times d_\ell^{\mathrm{in}}}$ and uses the effective weight

$$W_\ell = \bar{W}_\ell + s\, B_\ell A_\ell, \qquad s = \alpha/r. \tag{2}$$

B_ℓ is initialized to zero and A_ℓ with Kaiming-uniform values, so $W_\ell = \bar{W}_\ell$ at the start. We set $\alpha = 2r$, which gives the same scaling $s = 2$ for every rank; products trained at different ranks are therefore directly comparable, a prerequisite for aggregating them. IMU encoders are largely convolutional, so LoRA is defined for convolutions as well: a kernel $\mathcal{K} \in \mathbb{R}^{C_{\mathrm{out}} \times C_{\mathrm{in}} \times k_1 \times \cdots}$ is treated as the matrix $\mathbb{R}^{C_{\mathrm{out}} \times (C_{\mathrm{in}} k_1 \cdots)}$. The low-rank branch is computed as a convolution with r output channels and the original kernel geometry (stride, dilation and padding mode), followed by a 1×1 convolution back to C_{out} channels. All factors are stored as two-dimensional matrices irrespective of the layer type, which makes the server-side aggregation independent of the encoder architecture. Without LoRA dropout, the forward pass uses the merged kernel of Eq. (2); this is mathematically identical, autograd still only updates A_ℓ and B_ℓ, and it avoids two extra convolutions per layer.

4) Local normalization ③: The affine parameters (γ, β) of every LayerNorm and BatchNorm layer, and the running statistics (μ, σ^2) of BatchNorm layers, form ψ_k. They are trained and kept on the device, in the spirit of FedBN [1]. When the normalization parameters are not trained (methods V1–V4 in Table VIII), BatchNorm layers are kept in inference mode so that their pretrained statistics do not drift.

5) Shared projection head ④: $g_\theta(\mathbf{z}) = \mathrm{ReLU}\big(W_g\,\mathrm{Dropout}(\mathrm{LN}(\mathbf{z})) + \mathbf{b}_g\big)$ maps the encoder feature $\mathbf{z} \in \mathbb{R}^{d_z}$ to an embedding $\mathbf{e} \in \mathbb{R}^D$ with $D = 64$. The leading LayerNorm puts the very different feature scales of the four encoders ($d_z \in \{72, 256, 512, 1024\}$) on a common footing. Because g_θ is federated, all users embed activities into the same space, which is what makes prototypes comparable across users.

6) Personal classifier ⑤: $h_k(\mathbf{e}) = W_k\mathbf{e} + \mathbf{b}_k$ with $W_k \in \mathbb{R}^{C\times D}$ is kept local. A personal classifier may legitimately encode the user's own label distribution (a person who never cycles should rarely predict cycling), which a shared classifier cannot do.

7) Class prototypes ⑥: After local training, client k computes the mean embedding of each class it observes,

$$\mathbf{p}_{k,c} = \frac{1}{|\mathcal{I}_{k,c}|} \sum_{i\in\mathcal{I}_{k,c}} \mathbf{e}_{k,i}, \tag{3}$$

on a random subset $\mathcal{I}_{k,c}$ of at most 512 of its training windows, and reports $\mathbf{p}_{k,c}$ together with its true class count $n_{k,c}$.

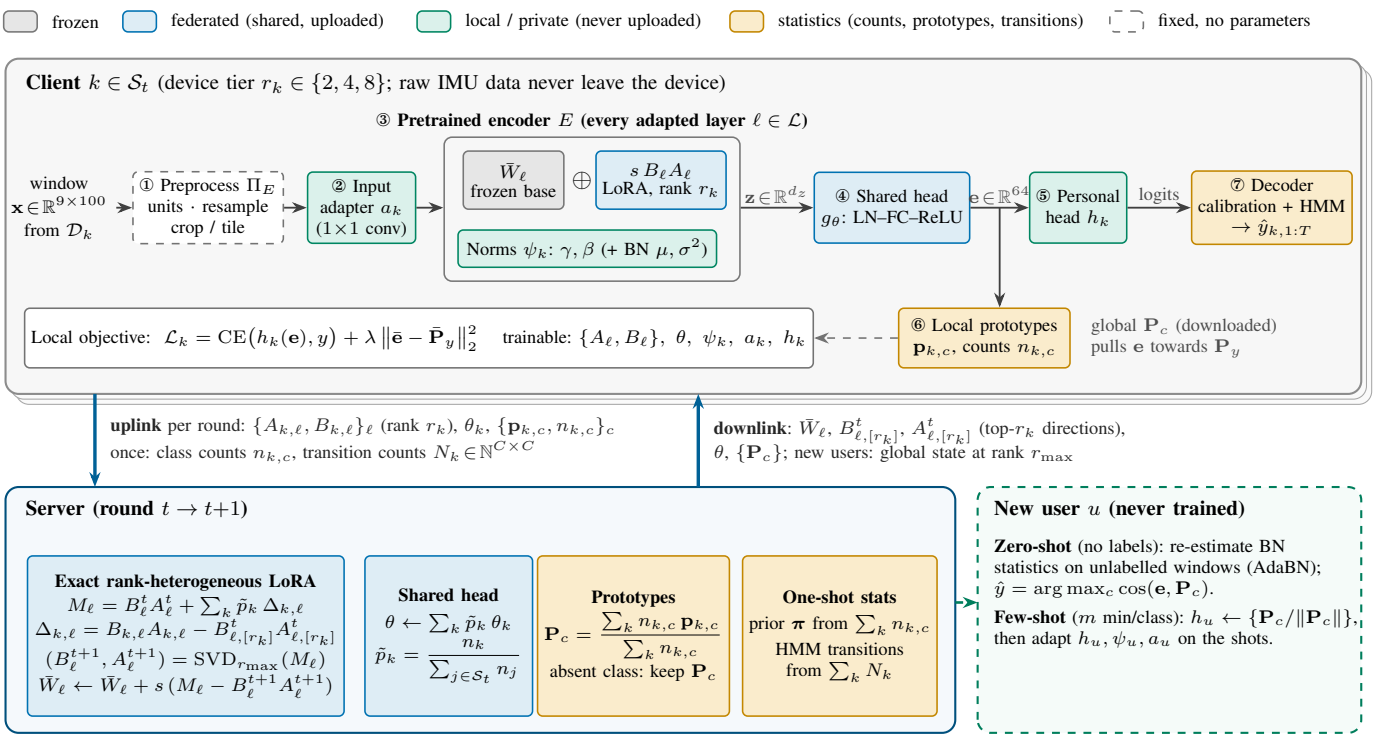


Fig. 1. FedPEFT-IMU architecture. Each client runs the same seven-component model ①–⑦ on top of a frozen pretrained IMU encoder. Parameters are partitioned by role: the base weights are frozen (grey); the low-rank adapters and the projection head are federated (blue) and trained at a device-specific rank r_k; the input adapter, the normalization layers and the classifier stay on the device (green); class prototypes and one-shot label statistics are the only additional information that is shared (amber). The server aggregates LoRA updates exactly in delta form, re-factorizes them by a truncated SVD and folds the remainder into the frozen weights (Fig. 2). New users are served without any training through the global prototypes.

The full forward pass of client k is

$$\mathbf{e} = g_\theta\Big(E\big(a_k(\Pi_E(\mathbf{x}));\ \{\bar{W}_\ell + s\,B_\ell A_\ell\}_{\ell\in\mathcal{L}},\ \psi_k\big)\Big), \qquad \hat{\mathbf{y}} = h_k(\mathbf{e}). \tag{4}$$

D. Local Objective

Given the global prototypes $\{\mathbf{P}_c\}_{c\in\mathcal{C}_P}$ of the previous round ($\mathcal{C}_P$ is the set of classes that already have one), client k minimizes

$$F_k(\Theta, \phi_k) = \frac{1}{n_k} \sum_{(\mathbf{x},y)\in\mathcal{D}_k^{\mathrm{tr}}} \Big[\mathrm{CE}\big(h_k(\mathbf{e}), y\big) + \lambda\, \mathbf{1}[y \in \mathcal{C}_P]\, \big\|\bar{\mathbf{e}} - \bar{\mathbf{P}}_y\big\|_2^2\Big], \tag{5}$$

over the trainable set $\{A_\ell, B_\ell\}_{\ell\in\mathcal{L}} \cup \{\theta\} \cup \phi_k$, where bars denote ℓ_2-normalized vectors and $\lambda = 0.5$. The prototype term pulls each embedding towards the population-level center of its class and thereby counteracts the drift of the shared representation towards the local label distribution [5]. We compute it on the unit sphere, where

$$\big\|\bar{\mathbf{u}} - \bar{\mathbf{v}}\big\|_2^2 = 2 - 2\cos(\mathbf{u}, \mathbf{v}) \in [0, 4]. \tag{6}$$

Unlike the unnormalized distance, Eq. (6) cannot be decreased by shrinking all embeddings, and its magnitude does not depend on the feature scale of the encoder, so a single λ serves all four encoders. The classifier is trained with plain cross-entropy: h_k is personal, so the Bayes-optimal rule *for client k* includes its own prior $\boldsymbol{\pi}_k$ and no re-balancing is needed (Section III-G discusses the shared-classifier case).

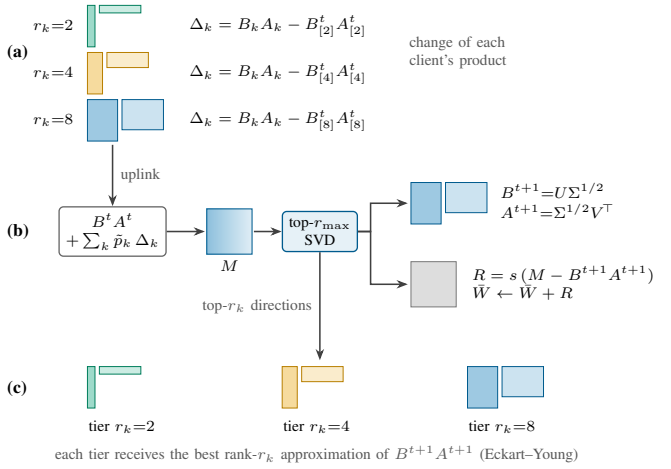


Fig. 2. Exact rank-heterogeneous aggregation for one adapted layer (glyph widths are proportional to the rank). (a) Each client uploads factors of its own rank r_k; the server forms the change Δ_k of the client's product relative to what the client received. (b) The weighted mean change is added to the current global product; the result M is re-factorized by a truncated SVD at rank $r_{\max}$, and the remainder R is folded into the frozen weight, so the effective model equals the aggregate of Proposition 1. (c) A device of rank r_k receives the leading r_k singular directions.

E. Exact Rank-Heterogeneous LoRA Aggregation

1) Why naive averaging fails: Averaging the factors separately (FedAvg on A and B) does not average the models, because the product of averages differs from the average of products.

Lemma 1 (Bias of factor averaging): Let $\bar{B} = \sum_k \tilde{p}_k B_k$ and $\bar{A} = \sum_k \tilde{p}_k A_k$ with $\sum_k \tilde{p}_k = 1$. Then

$$\bar{B}\bar{A} - \sum_k \tilde{p}_k B_k A_k = -\sum_k \tilde{p}_k \, (B_k - \bar{B})(A_k - \bar{A}). \quad (7)$$

Proof: Expanding the right-hand side gives $-\sum_k \tilde{p}_k B_k A_k + \bar{B}\bar{A} + \bar{B}\bar{A} - \bar{B}\bar{A}$. ∎

The error is a weighted cross-covariance of the client factors. It vanishes only if the clients' factors do not vary, and it grows precisely when the clients' data differ, i.e., under (H1)–(H2). FedEx-LoRA [6] removes it for equal ranks by adding the residual $s(\sum_k \tilde{p}_k B_k A_k - \bar{B}\bar{A})$ to the frozen weight. With heterogeneous ranks, however, two further problems appear: the factors have different shapes, and a low-rank device cannot represent, and therefore cannot update, the trailing directions of the global adapter.

2) Delta-form aggregation: The server keeps, for every $\ell \in \mathcal{L}$, the base $\bar{W}^t_\ell$ (initially Ω_0) and global factors (B^t_ℓ, A^t_ℓ) at rank $r_{\max} = \max \mathcal{R}$. We drop the layer index when no confusion arises. Client k receives the truncated factors $(B^t_{[r_k]}, A^t_{[r_k]})$, trains them, and uploads (B_k, A_k) of rank r_k. The server computes the change of each client's product relative to what that client received,

$$\Delta_k = B_k A_k - B^t_{[r_k]} A^t_{[r_k]}, \quad (8)$$

adds the weighted mean change to the current global product, re-factorizes the result at rank $r_{\max}$ and folds the remainder into the frozen weight:

$$M = B^t A^t + \sum_{k \in \mathcal{S}_t} \tilde{p}_k \, \Delta_k, \quad (9)$$

$$U \Sigma V^\top = \mathrm{SVD}_{r_{\max}}(M),$$

$$(B^{t+1}, A^{t+1}) = \big(U\Sigma^{1/2}, \ \Sigma^{1/2} V^\top\big), \quad (10)$$

$$\bar{W}^{t+1} = \bar{W}^t + s\,\big(M - B^{t+1}A^{t+1}\big). \quad (11)$$

Here $\tilde{p}_k = n_k / \sum_{j \in \mathcal{S}_t} n_j$ and $\mathrm{SVD}_{r_{\max}}$ returns the leading $r_{\max}$ singular triplets. Directions whose singular value is numerically zero ($\sigma_j < 10^{-6}\sigma_1$) would leave both factors at zero and block their gradients; for these we re-initialize the rows of A^{t+1} randomly and keep the columns of B^{t+1} at zero, which leaves the product unchanged. For large matrices ($\min(d^{\mathrm{out}}, d^{\mathrm{in}}) > 4r_{\max}$) a randomized SVD [7] with 8 oversampling vectors and 4 power iterations is used; as shown next, this does not affect exactness, because whatever the factors miss is kept in the residual.

Proposition 1 (Exactness): Let $W^t = \bar{W}^t + sB^tA^t$ be the effective global weight, and let $W^{\mathrm{start}}_k = \bar{W}^t + s\,B^t_{[r_k]}A^t_{[r_k]}$ and $W^{\mathrm{end}}_k = \bar{W}^t + sB_kA_k$ be client k's effective weight before and after local training. For any ranks $\{r_k\}$ and any factorization returned in Eq. (10),

$$W^{t+1} = W^t + \sum_{k \in \mathcal{S}_t} \tilde{p}_k \big(W^{\mathrm{end}}_k - W^{\mathrm{start}}_k\big). \quad (12)$$

Proof: By Eq. (11), $W^{t+1} = \bar{W}^{t+1} + sB^{t+1}A^{t+1} = \bar{W}^t + sM$. Substituting Eq. (9) gives $W^{t+1} = \bar{W}^t + sB^tA^t + \sum_k \tilde{p}_k \, s\Delta_k = W^t + \sum_k \tilde{p}_k \, s\Delta_k$, and $s\Delta_k = W^{\mathrm{end}}_k - W^{\mathrm{start}}_k$ by Eq. (8). ∎

Corollary 1 (Reduction to FedAvg): If all clients use $r_k = r_{\max}$, then $W^{\mathrm{start}}_k = W^t$ for every k, and Eq. (12) becomes $W^{t+1} = \sum_k \tilde{p}_k W^{\mathrm{end}}_k$: the effective models are averaged exactly as FedAvg [8] would average fully fine-tuned layers, and the rule coincides with FedEx-LoRA [6].

In other words, the server performs FedAvg on the *effective-weight updates* of the clients, regardless of their ranks and of how the update is factorized. Over rounds, the accumulated residual lets the global update grow beyond rank $r_{\max}$, while every device trains only $r_k(d^{\mathrm{in}} + d^{\mathrm{out}})$ parameters per layer.

3) Why the delta form matters: A natural alternative, used in spirit by product-space methods such as FlexLoRA [9], averages the zero-padded products, $M' = \sum_k \tilde{p}_k B_k A_k$. The following property shows that this alternative is biased under rank heterogeneity, whereas the delta form is not.

Proposition 2 (Consistency): Suppose that, after an aggregation, $B^tA^t = \sum_{j=1}^{r_{\max}} \sigma_j \mathbf{u}_j \mathbf{v}_j^\top$ is in the balanced SVD form of Eq. (10), and that no client changes its factors ($B_k = B^t_{[r_k]}$, $A_k = A^t_{[r_k]}$). Then the delta-form rule returns $W^{t+1} = W^t$, whereas product averaging returns

$$M' = \sum_{j=1}^{r_{\max}} \varrho_j \, \sigma_j \mathbf{u}_j \mathbf{v}_j^\top, \qquad \varrho_j = \sum_{k \in \mathcal{S}_t:\ r_k \geq j} \tilde{p}_k \ \leq 1. \quad (13)$$

Proof: With unchanged factors, $\Delta_k = 0$ for all k, so $M = B^tA^t$ and $W^{t+1} = W^t$ by Proposition 1. For product averaging, the balanced form gives $B^t_{[r]}A^t_{[r]} = \sum_{j \leq r} \sigma_j \mathbf{u}_j \mathbf{v}_j^\top$; summing over clients with weights $\tilde{p}_k$ and exchanging the sums yields Eq. (13). ∎

Under product averaging, the j-th direction of the global adapter is therefore multiplied by ϱ_j in every round, independently of the data. With tiers drawn uniformly from $\{2, 4, 8\}$, $\mathbb{E}[\varrho_j] = 1,\ \frac{2}{3},\ \frac{1}{3}$ for $j \in \{1, 2\}$, $\{3, 4\}$ and $\{5, \ldots, 8\}$: what the high-tier devices learn is diluted geometrically by the low-tier devices that never hold those directions. In the delta form, a client that does not hold a direction contributes a zero *change* to it rather than a zero *value*. Finally, because (B^t, A^t) is in balanced SVD form, the truncation sent to a rank-r_k device is the best rank-r_k approximation of the global adapter in the Frobenius and spectral norms (Eckart–Young–Mirsky [10]): weak devices receive the most informative part of the global update.

F. Aggregation of the Head, Prototypes and Statistics

The projection head is averaged as $\theta^{t+1} = \sum_{k \in \mathcal{S}_t} \tilde{p}_k \theta_k$. Prototypes are aggregated with the class counts as weights,

$$\mathbf{P}_c = \frac{\sum_{k \in \mathcal{S}_t} n_{k,c} \, \mathbf{p}_{k,c}}{\sum_{k \in \mathcal{S}_t} n_{k,c}}, \quad (14)$$

so a client contributes to a class only if it observes that class; a class that no participant observes in a round keeps its previous prototype. In addition, every client uploads two statistics once, before training: its class counts $\{n_{k,c}\}_c$ and its activity-transition counts $N_k \in \mathbb{N}^{C \times C}$, computed from its chronologically ordered training labels. The server forms the global prior $\pi_c \propto \sum_k n_{k,c}$ and the transition matrix of the temporal decoder (Section III-H). Together these statistics cost $C^2 + C = 42$ integers per client.

G. Learning under Label Skew

1) Why rare classes vanish with a shared classifier: With a shared classifier f and plain cross-entropy, the gradient with respect to the logit of class c is $q_c - \mathbf{1}[y = c]$, where $q = \mathrm{softmax}(f(\mathbf{x}))$. On a client that never observes c, every sample therefore lowers the logit of c. When a class is absent on most clients, averaging their updates erases it from the global model. In our earlier experiments on the free-living Capture-24 dataset [11], where three quarters of the users never cycle, 35 of the 168 per-class F1 scores of the federated models were zero without the remedies described below. HARBox is far less skewed (Section IV-D), but some users still lack one or two activities.

2) FedPEFT-IMU: The proposed design avoids this structurally: the classifier is personal (⑤), so no client's decision rule is pulled towards other clients' priors. The only shared class-specific quantities are the prototypes, and Eq. (14) averages each prototype only over the clients that have the class.

3) Shared-classifier methods: So that the comparison with methods that share the classifier (V0–V4 in Table VIII) is not dominated by the class-imbalance problem, those methods are equipped with three standard remedies. First, every client trains with a logit-adjusted (balanced-softmax) loss [12]–[14] that uses its *own* prior,

$$\ell(\mathbf{x}, y) = \mathrm{CE}\big(f(\mathbf{x}) + \tau \log \boldsymbol{\pi}_k,\ y\big), \qquad \tau = 1, \tag{15}$$

where $\boldsymbol{\pi}_k$ is floored at 10^{-4}. The gradient for an absent class becomes $q'_c \propto \pi_{k,c}^{\tau} e^{f_c} \approx 10^{-4} e^{f_c}$, so absent classes are no longer pushed down, and f learns class-balanced scores. Second, the rows of the shared classifier are aggregated class-wise [15], [16],

$$[W]_c^{t+1} = \frac{\sum_{k \in \mathcal{S}_t} n_{k,c} [W_k]_c}{\sum_{k \in \mathcal{S}_t} n_{k,c}} \quad \text{(the same for } b_c\text{)}, \tag{16}$$

and a row that no participant observes keeps its value. Third, decisions are calibrated with the global prior (Eq. (17)). Full-model methods (FedAvg and V0) also use a larger learning rate for the heads than for the body (discriminative learning rates), without which the rows of rare classes barely move within the training budget.

H. Inference

1) Enrolled users: FedPEFT-IMU predicts $\hat{y} = \arg\max_c [h_k(\mathbf{e})]_c$ with the user's rank, local parts and personal head. A shared classifier trained with Eq. (15) outputs class-balanced scores, which we convert into decisions with a tunable prior strength,

$$\hat{y} = \arg\max_c \ f_c(\mathbf{x}) + \tau_{\mathrm{inf}} \log \pi_c. \tag{17}$$

$\tau_{\mathrm{inf}} = 0$ gives balanced decisions and $\tau_{\mathrm{inf}} = 1$ the accuracy-optimal decisions under $\boldsymbol{\pi}$; macro-F1 is best in between. We select $\tau_{\mathrm{inf}} = 0.5$ from $\{0, 0.25, 0.5, 0.75, 1\}$ on the validation users and keep it fixed for all methods and encoders.

2) Federated temporal decoding (continuous streams): In free-living data, activities last minutes to hours, whereas the classifier sees a single window. Following the Capture-24 benchmark [11], [17], FedPEFT-IMU can therefore decode each user's chronological sequence with a hidden Markov model (HMM) whose hidden states are the activities. The transition matrix is estimated from the aggregated counts with add-one smoothing, $\Gamma_{ij} = (1 + \sum_k N_{k,ij}) / \sum_{j'} (1 + \sum_k N_{k,ij'})$, so it is learned federatedly from C^2 integers per client without sharing any label sequence. The emission score of window $\mathbf{x}$ for state c is the class-balanced score,

$$\log p(\mathbf{x} \mid c) \simeq \log \mathrm{softmax}\big(f(\mathbf{x}) - (1 - \tau) \log \tilde{\boldsymbol{\pi}}\big)_c, \tag{18}$$

with $\tau = 1$ and $\tilde{\boldsymbol{\pi}} = \boldsymbol{\pi}$ for a logit-adjusted shared classifier. For a personal classifier ($\tau = 0$) we divide by the client's prior shrunk halfway towards the global one, $\tilde{\boldsymbol{\pi}} = \frac{1}{2}\boldsymbol{\pi}_k + \frac{1}{2}\boldsymbol{\pi}$; dividing by $\boldsymbol{\pi}_k$ alone would boost classes that the client has never observed by up to $-\log 10^{-4} \approx 9$ nats. The Viterbi algorithm [18] returns the most likely activity sequence in $\mathcal{O}(|\mathcal{D}_k^{\mathrm{te}}|\, C^2)$ time. Sequences are split wherever two consecutive windows are more than three nominal spacings apart, so the decoder never smooths across recording gaps. Decoding runs on the device; it uses only the shared Γ and $\boldsymbol{\pi}$.

3) New users: A user u who never participated downloads the global state (rank $r_{\max}$) and the prototypes. Its local parts must be initialized. The pretrained normalization and adapter do not match the representation that the federated LoRA and head were trained with, because every client trained its own. Therefore, once at the end of training, every client uploads its local normalization parameters and input adapter (not its personal classifier), and the server keeps their n_k-weighted average $\bar{\psi}, \bar{a}$ as the initialization for new users. On LIMU-BERT this costs a one-time upload of 2 kB per client and raises the new-user scores at every label budget (Section V-E). *Zero-shot*: encoders with BatchNorm additionally re-estimate their normalization statistics on the user's *unlabelled* windows (AdaBN [19]); the user is then served by the nearest-prototype rule

$$\hat{y} = \arg\max_{c \in \mathcal{C}_P} \ \cos\big(\mathbf{e}, \mathbf{P}_c\big). \tag{19}$$

Transductive zero-shot (optional). The global prototypes describe the federation, not the new user. Before serving, the device can move them towards its own unlabelled windows $\{\mathbf{z}_i\}$ (unit-norm embeddings of the same windows AdaBN uses) with T soft k-means steps anchored at the global prototypes $\hat{\mathbf{P}}_c^{(0)} = \mathbf{P}_c / \|\mathbf{P}_c\|$:

$$\begin{aligned} w_{ic} &= \mathrm{softmax}_{c \in \mathcal{C}_P}\big(\mathbf{z}_i^{\top} \hat{\mathbf{P}}_c^{(t)} / \vartheta\big), \\ \hat{\mathbf{P}}_c^{(t+1)} &= \mathbf{m}_c / \|\mathbf{m}_c\|, \quad \mathbf{m}_c = \textstyle\sum_i w_{ic}\, \mathbf{z}_i + \kappa\, \hat{\mathbf{P}}_c^{(0)}, \end{aligned} \tag{20}$$

and then applies Eq. (19) with $\hat{\mathbf{P}}^{(T)}$. The anchor weight κ, in windows, keeps a class the user rarely performs close to its global prototype; without it ($\kappa = 0$) the clusters drift away from their classes. This is the prototype counterpart of test-time classifier adjustment [20]: it uses no labels, no communication and no gradient step. We use $T = 10$, $\vartheta = 0.1$ and $\kappa = 10$, selected on the validation users. Applied to the normalized class weights of a shared classifier, the same

update lowers the validation zero-shot score, so the baselines keep their own classifier. *Few-shot*: with m minutes of labels per activity, a new personal head is initialized at the zero-shot solution, $[W_u]_c = \mathbf{P}_c/\|\mathbf{P}_c\|$ and $b_{u,c} = 0$ (with the refined prototypes when the transductive rule is used), since $\arg\max_c \cos(\mathbf{e}, \mathbf{P}_c) = \arg\max_c \mathbf{P}_c^\top \mathbf{e}/\|\mathbf{P}_c\|$. The head, the norms and the input adapter $\{h_u, \psi_u, a_u\}$ are then fine-tuned on the shots. BatchNorm statistics stay fixed at their AdaBN estimates during this fit: the scale and shift of ψ_u adapt to the labels, but the statistics come from all of the user's unlabelled windows, not from a batch of a few shots. Because the shots are roughly class-balanced, unlike a real day, this fit uses Eq. (15) with the prior of the shots, and decisions use Eq. (17).

I. Training-Time Augmentation

The same device can be held at many angles, and different devices use different axis conventions. During local training, each window is therefore transformed, with probability p, by one random rotation $R \in SO(3)$ (uniform axis, angle up to $\theta_{\max}$) and a random gain g: $\mathbf{x}_s \leftarrow g\,R\,\mathbf{x}_s$ for every sensor triplet s (accelerometer, gyroscope, magnetometer). The same R is applied to all triplets, as for a rotated phone. The transformation acts on the stored windows before Π_E, so it is identical for every encoder and method, and it costs one batched $C \times C$ matrix product per mini-batch.

J. Training Procedure

Algorithm 1 summarizes training. Each device is assigned a rank tier once, and each round consists of download, local training of the trainable roles, prototype computation, upload of the federated roles and statistics, and server aggregation. Because only a fraction ρ of the clients participates in a round, the personal parts of a client may lag behind the latest global representation. After the last round, every client therefore performs a short personalization pass (8 steps) that updates only ϕ_k against the final global model, without any communication.

K. Communication, Memory and Computation

1) Uplink: Per round, client k uploads

$$U_k = \underbrace{r_k \textstyle\sum_{\ell\in\mathcal{L}} \left(d_\ell^{\text{in}} + d_\ell^{\text{out}}\right)}_{\text{LoRA}} + \underbrace{|\theta|}_{\text{head}} + \underbrace{C(D+1)}_{\text{prototypes, counts}} \qquad (21)$$

values, which is linear in the rank and independent of the size of the frozen encoder. Full fine-tuning uploads $|\Omega_0| + |\theta| + |h|$ values. Table III lists the resulting budgets for the four encoders on HARBox together with the measured uplink: FedPEFT-IMU sends $8\times$ (LIMU-BERT, the smallest encoder) to $95\times$ fewer bytes per update than full fine-tuning, and the factor grows with the size of the encoder.

2) Downlink: In addition to the truncated factors, the head and the prototypes, exact aggregation changes the frozen weights $\bar{W}_\ell$, which a client must receive before it trains again. In our accounting we conservatively charge the full dense $\bar{W}_\ell$ of every adapted layer at each download. This term can be reduced substantially: if the truncated SVD in Eq. (10) is exact, the per-round residual in Eq. (11) has rank at most $\sum_{k\in\mathcal{S}_t} r_k$, because $\operatorname{rank}(M) \le r_{\max} + \sum_{k\in\mathcal{S}_t} r_k$, and it can therefore be transmitted in factored form. We report the conservative figure and return to this point in the discussion.

Algorithm 1: FedPEFT-IMU training

Input : clients $k = 1..K$ with $\mathcal{D}_k^{\text{tr}}$; tiers $\mathcal{R}$; rounds T; rate ρ; local steps S; weight λ; pretrained Ω_0
Output : global state $(\{\bar{W}_\ell, B_\ell, A_\ell\}_\ell, \theta, \{\mathbf{P}_c\}, \boldsymbol{\pi}, \Gamma)$; local parts $\{\phi_k\}$

Server: $\bar{W}_\ell \leftarrow \Omega_{0,\ell}$, $B_\ell \leftarrow 0$, $A_\ell \leftarrow \text{Kaiming}(r_{\max})$, $\theta \leftarrow$ init, $\mathcal{C}_P \leftarrow \emptyset$;
for *each client* k **do**
  $r_k \sim \text{Uniform}(\mathcal{R})$;
  $\phi_k \leftarrow$ (identity a_k, pretrained ψ_k, random h_k);
  send $\{n_{k,c}\}_c$ and N_k to the server ; *// once*
Server: build the prior $\boldsymbol{\pi}$ and the HMM transitions Γ;
for $t = 0, \dots, T-1$ **do**
  $\mathcal{S}_t \leftarrow$ random subset of $\lceil \rho K \rfloor$ clients;
  for $k \in \mathcal{S}_t$ **in parallel do**
    receive $\{\bar{W}_\ell, B_{\ell,[r_k]}, A_{\ell,[r_k]}\}_\ell$, θ, $\{\mathbf{P}_c\}_{c\in\mathcal{C}_P}$;
    S AdamW steps on F_k (Eq. (5)) w.r.t. $\{A_\ell, B_\ell\}, \theta, \phi_k$;
    compute $\mathbf{p}_{k,c}$ by Eq. (3); keep ϕ_k on the device;
    upload $\{A_{k,\ell}, B_{k,\ell}\}_\ell$, θ_k, $\{\mathbf{p}_{k,c}, n_{k,c}\}_c$, n_k;
  Server: for every $\ell \in \mathcal{L}$ apply Eqs. (8) to (11);
  Server: $\theta \leftarrow \sum_k \tilde{p}_k \theta_k$; prototypes by Eq. (14);
for *each client* k **do** *// final personalization, no communication*
  8 steps on F_k w.r.t. ϕ_k only, at the final global state;

3) Client memory and computation: Only LoRA factors, the heads and ϕ_k require gradients and optimizer states, i.e., three additional floating-point values per trainable parameter for AdamW [21]. Activation memory still scales with the encoder, so the memory saving is largest when the optimizer state dominates (large convolutional encoders) and modest for small Transformers. Prototype computation needs one extra forward pass over at most 512 windows per round.

4) Server computation: Forming M_ℓ costs $\mathcal{O}\big(d_\ell^{\text{out}} d_\ell^{\text{in}} \sum_{k\in\mathcal{S}_t} r_k\big)$, and the randomized SVD costs $\mathcal{O}\big(d_\ell^{\text{out}} d_\ell^{\text{in}} (r_{\max}+8)\big)$ per layer and round. The server streams the uploads into running sums, so its memory does not grow with the number of participating clients.

L. Privacy Considerations

Raw windows, labels, label sequences, local normalization statistics, input adapters and personal classifiers never leave the device. The uploads are the LoRA factors and the projection head (as in any PEFT-based federated method), C class-mean embeddings in a 64-dimensional space and their counts, and, once, $C + C^2$ label statistics. Prototypes and counts reveal which activities a user performed, so they are as sensitive as a coarse activity diary; they can be protected by adding calibrated Gaussian noise to $\mathbf{p}_{k,c}$ and $n_{k,c}$ or by aggregating them with secure aggregation [26], because Eq. (14) is a ratio of two sums. The products $B_k A_k$ in Eq. (9) are also sums over clients, but secure aggregation of their dense form would forgo the uplink savings. A formal privacy analysis is beyond the scope of this work.

TABLE III
PARAMETER BUDGET PER ENCODER ON HARBOX AND MEASURED COMMUNICATION PER CLIENT UPDATE (FP32 SERIALIZATION, NO COMPRESSION). $|\Omega_0|$: FROZEN ENCODER WEIGHTS; LORA: TRAINABLE ADAPTER PARAMETERS AT RANK $r = 2/4/8$; $|\theta|$: SHARED HEAD; $|\phi_k|$: LOCAL PARAMETERS (NORM AFFINE, ADAPTER, CLASSIFIER).

Encoder	LoRA targets	$\lvert\Omega_0\rvert$	LoRA (r=2/4/8)	$\lvert\theta\rvert$	$\lvert\phi_k\rvert$	Uplink FedPEFT-IMU r=2/4/8 (MB)	Uplink full FT (MB)	Ratio†
LIMU-BERT [22]	W_q, W_v (shared layers)	45.4 k	0.6/1.2/2.3 k	4.8 k	0.8 k	0.023/0.025/0.030	0.204	7.9×
ssl-wearables [23]	all Conv1d kernels	10.45 M	60/119/239 k	67.6 k	12.0 k	0.511/0.749/1.227	42.15	51.9×
UniMTS [24]	graph + temporal convs	5.17 M	46/91/182 k	33.9 k	8.9 k	0.319/0.501/0.866	20.89	37.6×
HALO‡ [25]	W_q, W_v (all blocks)	4.60 M	14/28/55 k	17.0 k	9.6 k	0.124/0.180/0.290	18.51	95.3×

†Mean uplink of full fine-tuning divided by the mean uplink of FedPEFT-IMU over all client updates (tiers drawn uniformly). ‡Our re-implementation of the HALO encoder, pretrained self-supervised on the pool users (Section IV-E).

TABLE IV
HARBOX ACTIVITIES AFTER WINDOWING (2 S WINDOWS, 50 HZ, 9 AXES), OVER ALL 120 USERS. USERS: USERS WITH AT LEAST ONE WINDOW OF THE ACTIVITY.

Activity	Description	Windows	%	Users
Call	phone held at the ear during a call	6,514	19.1	117
Hop	hopping on the spot	6,857	20.1	119
Typing	typing on the phone	6,453	18.9	118
Walk	walking	8,089	23.7	120
Wave	waving the phone-holding hand	6,202	18.2	117
Total		34,115	100.0	120

IV. DATASET AND EXPERIMENTAL PROTOCOL

A. The HARBox Dataset

We evaluate on HARBox [27], the largest of the federated HAR datasets released with ClusterFL. Unlike laboratory collections, it was *crowd-sourced*: volunteers installed an Android application on their own smartphones and recorded five activities of daily life, labelling each recording themselves by pressing "start" and "end" in the application (Table IV). After invalid and repeated submissions were removed, the data of 121 users aged 17–55 years remained, recorded with 77 different smartphone models. Each recording contains the 9-axis inertial signals of the phone (tri-axial accelerometer in $\mathrm{m\,s^{-2}}$, gyroscope in $\mathrm{rad\,s^{-1}}$ and magnetometer in μT), resampled by the authors to 50 Hz and cut into 2 s windows with a 1 s hop, so consecutive windows of a recording overlap by 50 %. One user has no windows. The remaining 120 users contribute 34,115 windows (about 9.5 h of distinct signal). Users and phones are the natural unit of federation, so every user is one client and no synthetic partition is needed. The heterogeneity that federated HAR must handle (different devices, grips, body sizes and movement styles) is present by construction.

B. Preprocessing

We use the windows exactly as released: each window is a 9×100 array in physical units, ordered by its start time, and assigned the activity of the file it comes from. We also reconstruct the recording session of each window from the timestamps: a new session starts wherever two consecutive windows of an activity start more than 1.5 s apart. The median user contributes 46 windows per activity, i.e., less than one minute of recording, in a median of five sessions. No resampling or filtering is applied at storage time. The encoder-specific preprocessing Π_E (Section III-C) converts units, resamples to the encoder's rate and fits the window length (Table VII). Only the training windows are augmented (Section IV-H).

C. Evaluation Protocols

Two protocols are used (Table V).

Protocol A (published). To compare with published results, we reproduce the protocol of FedCHAR [28]: every user is a client, draws a random number of 50 to 500 of its windows, and splits them at random into 72 % training, 8 % validation and 20 % test windows. Training runs for 100 rounds with half of the users per round, and the metric is the mean test accuracy over users. Because consecutive windows overlap by 50 %, however, a random split places at least one neighbour of most test windows in the training set. We measured that 79.7 % of the test windows share half of their samples with a training window (Fig. 3c). Protocol A therefore rewards memorizing a user's recordings rather than recognizing the activity, and we use it only for the comparison with published numbers.

Protocol B (ours, leakage-free). The 120 users are randomly assigned, with a fixed seed, to 84 FL clients (70 %), 24 held-out users (20 %) and 12 pool users (10 %). Every activity of a client is split *in time*: the first 80 % of its windows are used for training and the last 20 % for testing. The window between the two parts is discarded, because it overlaps both. A plain chronological split of the whole recording would put entire activities into the test part, since HARBox records each activity in its own session. No test window overlaps a training window, and every activity of a client appears on both sides. The held-out users never take part in training. For each of their activities, the first half is the pool from which few-shot labels may be drawn and the second half is the test set (again with a one-window gap). The pool users provide unlabelled data for the self-supervised pretraining of HALO and serve as validation users for model selection. No test data are used for any choice. All main results of this paper use Protocol B. LIMU-BERT and the MLP baselines are repeated on two further user splits (seeds 1 and 2, Table XV).

D. Statistical Heterogeneity

HARBox differs from free-living datasets in where its heterogeneity lies (Table VI). Label skew is mild: 112 of the 120 users recorded all five activities, and a user's label

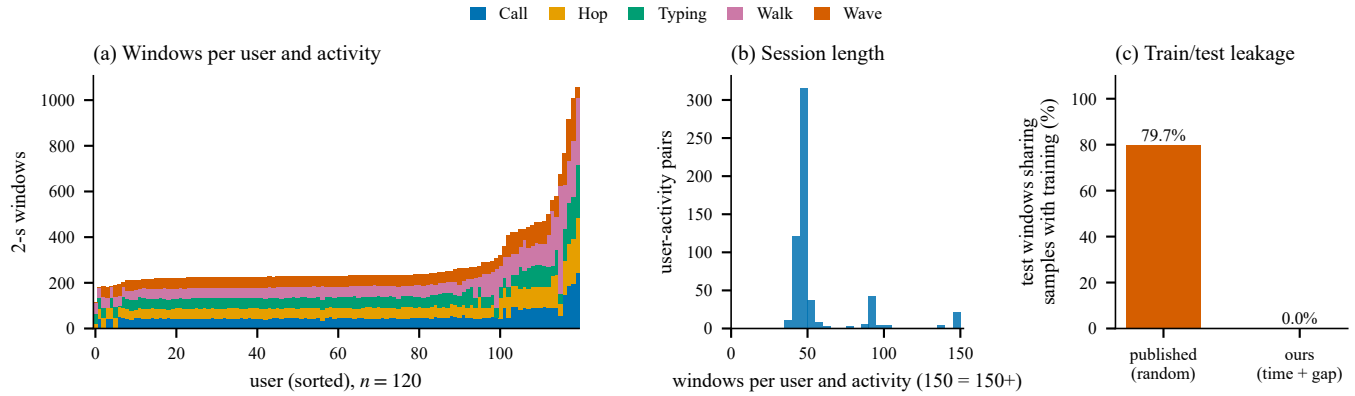


Fig. 3. HARBox at a glance. (a) Windows per user and activity: users differ in how long they recorded, not in which activities. (b) Length of the recording of one activity by one user, typically below one minute. (c) Share of test windows that overlap a training window by half of their samples. The random split of the published protocol leaks the large majority; our per-activity chronological split with a one-window gap leaks none.

TABLE V
THE TWO EVALUATION PROTOCOLS ON HARBOX (SEED 0). †POOL USERS' WINDOWS (VALIDATION).

	Protocol B (ours)	Protocol A (published)
Users: clients / held-out / pool	84 / 24 / 12	120 / 0 / 0
Windows per client	all	50–500 at random
Client split	per activity in time, 80/20, 1-window gap	random 72/8/20
Train / validation / test windows	19,408 / 2,822† / 4,614	16,637 / 1,848 / 4,624
Test windows overlapping a training window	0.0 %	79.7 %
New-user evaluation	24 users (zero-/few-shot)	–

TABLE VI
STATISTICAL HETEROGENEITY OF THE 120 HARBOX USERS. JSD: JENSEN–SHANNON DIVERGENCE IN BITS.

Statistic	Value
Windows per user (min / median / max)	115 / 230 / 1054
Quantity skew: coefficient of variation	0.53
Activities per user (histogram)	3: 1, 4: 7, 5: 112
Windows per user and activity, median	46 (≈0.8 min)
JSD(user label distribution ‖ global), mean	0.014

distribution diverges from the global one by only 0.014 bits on average. Quantity skew is substantial (115 to 1054 windows per user). The dominant heterogeneity is in the signals themselves. Seventy-seven phone models with different sensor ranges, noise levels and axis conventions are held, pocketed and waved by 120 different people, each recording every activity only once and for less than a minute. The class-conditional distributions $p_k(\mathbf{x} \mid y)$ therefore differ strongly between users (heterogeneity (H2) in Section III-A), and they can be learned only from very little data per user. This is the regime that the local normalization, the local input adapter and the personal classifier of FedPEFT-IMU are designed for. The label-skew remedies of Section III-G matter less here.

E. Pretrained Encoders

To test whether the conclusions depend on the backbone, we instantiate FedPEFT-IMU with four IMU encoders that differ in architecture (Transformer, residual CNN, spatio-temporal graph network, patch-channel Transformer), size ($51\,\mathrm{k}$ to $10.4\,\mathrm{M}$ frozen parameters), pretraining corpus and objective, and input format (Table VII). We add the multilayer perceptron of the published HARBox results [27], [28] as a from-scratch reference. Three encoders use their official checkpoints, loaded into re-implementations with identical parameter names (loading fails if less than $90\,\%$ of the tensors match). LIMU-BERT was pretrained on smartphone data with accelerometer and gyroscope [29], the closest match to HARBox. Its 120 position embeddings cover $6\,\mathrm{s}$ at $20\,\mathrm{Hz}$, of which a $2\,\mathrm{s}$ window uses the first 40. ssl-wearables expects $10\,\mathrm{s}$ at $30\,\mathrm{Hz}$, so the resampled $2\,\mathrm{s}$ window is repeated five times. UniMTS receives the accelerometer on the right-wrist joint of its 22-joint skeleton, the joint closest to a hand-held phone. HALO [25] has not released weights. We re-implemented its encoder at a reduced width ($d = 192$) and pretrained it with its stage-1 objectives (masked reconstruction of $50\,\%$ of the patches [30] and a patch-level InfoNCE loss [31]) on the unlabelled windows of the pool users, with all nine axes and $0.4\,\mathrm{s}$ patches. Because HALO tokenizes every channel separately, it is the only pretrained encoder that uses the magnetometer from the start. The input adapters of the others may learn to mix it in.

F. Compared Methods

Table VIII lists the federated methods, each defined by the roles of Table II. FedAvg trains the whole model from a random initialization. V0 and V1 are the upper-cost and lower-cost ways of using a pretrained encoder (full fine-tuning; a frozen encoder with trainable heads). V2 and V3 isolate the aggregation rule for homogeneous LoRA: factor averaging versus exact aggregation [6]. V4 adds rank heterogeneity with the delta-form aggregation of Section III-E, and it is the ablation of FedPEFT-IMU (V5) without local parts and prototypes. The MLP is trained with the baselines of the published HARBox

TABLE VII
ENCODERS AND THEIR HARBOX INPUT. INPUT: CHANNELS AFTER THE INPUT ADAPTER, SAMPLING RATE, SAMPLES (DURATION) AND UNITS, PRODUCED FROM THE STORED 9×100 WINDOW BY Π_E.

Encoder	Architecture	Pretraining data and objective	HARBox input	d_z	$\lvert\Omega_0\rvert$
LIMU-BERT [22]	Transformer, 4 weight-shared layers (ALBERT-style), $d = 72$, 4 heads, mean pooling	HHAR smartphones and smartwatches [29]; masked-span reconstruction (official checkpoint)	6 (acc+gyro), 20 Hz, 40 (2 s), g	72	51 k
ssl-wearables [23]	1-D ResNet-V2 (harnet10), 5 stages, 64–1024 channels, anti-aliased down-sampling	UK Biobank, 700 k person-days of wrist accelerometry; multi-task self-supervision	3 (acc), 30 Hz, 60 tiled to 300, g	1024	10.4 M
UniMTS [24]	ST-GCN [32], 10 blocks over a 22-joint skeleton graph, 64–512 channels	IMU synthesized from motion capture (HumanML3D [33]); contrastive text alignment	3 (acc) on wrist joint, 20 Hz, 40, $\mathrm{m\,s^{-2}}$	512	5.2 M
HALO [25] (re-impl.)	0.4-s patch $\times$ channel tokens, 8 alternating temporal/cross-channel attention layers, $d = 192$, query pooling	HARBox pool users (unlabelled, disjoint from all evaluation users); masked reconstruction + patch contrastive	9 (acc+gyro+mag), 50 Hz, 100, g	256	4.6 M
MLP [28]	flattened window, one hidden layer of 300 units (the published HARBox model)	none (trained from scratch)	9, 50 Hz, 100 (900 features)	300	–

results: FedAvg, purely local training, and Ditto [34], the personalized method on which FedCHAR builds. Because Ditto is the strongest of these, we also run it from the pretrained LIMU-BERT (Ditto, pretrained), which gives the personalized full-model baseline at the communication cost of V0. Every method uses the same augmentation, decoders and evaluation. Two centralized references are trained on the pooled training parts of the clients: *from scratch*, which defines the target accuracy, and *pretrained fine-tuning*, an upper reference without privacy constraints.

G. Evaluation Metrics

Recognition. Following the published HARBox results, the primary metric is the *mean per-user accuracy*, i.e., the test accuracy of every client averaged over clients. We also report the macro-F1 over the five activities, computed on the pooled test windows of all clients.

Fairness. We report the mean accuracy of the $\lceil 0.1K \rceil$ worst-served clients (*worst-10 %*) and the standard deviation of the per-client macro-F1 over the activities present in each client's test part.

Communication. Every message is an explicit set of tensors whose size is measured as the serialized size of uncompressed arrays (four bytes per real value, eight per integer), as in Flower's NumPy serialization [35]. It includes every tensor that is sent: factors, heads, residual weights, prototypes and counts. We report the uplink per client update (MB), the rounds to target (the first evaluated round whose macro-F1 reaches $90\,\%$ of the centralized from-scratch macro-F1 of the same encoder) and the cumulative uplink up to that round. No compression or quantization is applied.

Client cost. The client memory is the size of all model tensors plus the peak additional device memory allocated during a local update, measured with the CUDA allocator. The client time is the wall-clock time of one local update.

New users (Protocol B). On the 24 held-out users we report the mean per-user macro-F1 on their test parts: zero-shot (no labels) and few-shot with 1, 5 and 10 labelled windows (2, 10 and 20 s) per activity, drawn at random from their adaptation parts.

Personalized evaluation. Methods with a shared classifier are also scored after the common "FedAvg + local fine-tuning" step ("+FT"): every client fine-tunes only the classifier on its own training part (20 steps) and is scored with it. Local, Ditto and FedPEFT-IMU are personal by design. Temporal decoding (Section III-H) is not used on HARBox: every activity is recorded in its own session, so a sequence model would only learn that the label never changes within a recording.

H. Implementation Details

All methods run in one transparent federated simulator written in PyTorch 2.11 (Python 3.13). All windows reside in GPU memory, so a mini-batch is an index tensor and every method sees exactly the same data stream for a given seed. Experiments ran on a single NVIDIA GeForce GTX 1660 Ti (6 GB) in fp32. Table IX lists the hyperparameters. Because the 77 phone models are held in many different ways, every training window of every method is augmented with probability 0.8 by one random rotation of up to 30°, applied jointly to the accelerometer, gyroscope and magnetometer triplets, and by a random gain ($\mathcal{N}(1, 0.1^2)$). The three large encoders are 5–15 times slower per step than LIMU-BERT on this GPU and therefore train with a reduced budget (Table IX). All methods of an encoder share its budget, so methods are comparable within an encoder. All local updates use AdamW [21] with a fresh optimizer state in every round.

V. RESULTS

A. Comparison with Published Results

Table X places our models next to the published HARBox results under the published protocol. Our re-run of the published baseline, FedAvg with the 300-unit MLP, reaches a mean user accuracy of 0.797 (0.815 after local fine-tuning of its classifier), against 0.694 reported by FedCHAR. The difference comes from the training recipe shared by all our methods (AdamW,

TABLE VIII
FEDERATED METHODS. Ω: ENCODER WEIGHTS; g, h: SHARED AND CLASSIFIER HEADS; ψ: NORMALIZATION; a: INPUT ADAPTER; P: PROTOTYPES.

Method	Init.	Trained	Uploaded	Kept local	LoRA rank	Aggregation
Local	random	Ω, ψ, a, g, h	–	everything	–	none
FedAvg [8]	random	Ω, ψ, a, g, h	Ω, ψ, a, g, h	–	–	FedAvg
Ditto [34]	random	global + personal copy	Ω, ψ, a, g, h	personal copy	–	FedAvg + proximal personal model
V0 Full fine-tuning	pretrained	Ω, ψ, a, g, h	Ω, ψ, a, g, h	–	–	FedAvg
Ditto, pretrained	pretrained	global + personal copy	Ω, ψ, a, g, h	personal copy	–	FedAvg + proximal personal model
V1 Head only	pretrained	g, h	g, h	–	–	FedAvg
V2 LoRA, factor averaging	pretrained	LoRA, g, h	LoRA, g, h	–	8	avg. A, avg. B
V3 LoRA, exact	pretrained	LoRA, g, h	LoRA, g, h	–	8	FedEx-LoRA [6]
V4 Heterogeneous ranks	pretrained	LoRA, g, h	LoRA, g, h	–	2/4/8	delta form + SVD (Section III-E)
V5 FedPEFT-IMU	pretrained	LoRA, g, h, ψ, a	LoRA, g, P	h, ψ, a	2/4/8	delta form + SVD, prototypes

TABLE IX
HYPERPARAMETERS ON HARBOX (PROTOCOL B UNLESS STATED). BUDGET OF SSL-WEARABLES AND HALO / UNIMTS IN PARENTHESES WHERE IT DIFFERS.

Federated training	
Rounds T; clients per round	100 (60 / 40); 17 of 84 ($\rho = 0.2$)
Local work per round	2 epochs, capped at 8 (6 / 4) steps of batch 64
Learning rates	FedAvg 10^{-3}; V0 body 3×10^{-4}; LoRA, heads, ψ, a: 2×10^{-3}
Weight decay	10^{-4}
LoRA	$r = 8$ (V2, V3); tiers $\{2, 4, 8\}$ uniform (V4, V5); $\alpha = 2r$; dropout 0
Embedding D; prototype weight λ	64; 0.5 (prototypes on ≤ 512 windows)
Shared classifiers	$\tau = 1$ with client prior; class-wise rows; $\tau_{\text{inf}} = 0.5$
Ditto	proximal weight 0.1
Augmentation	rotation $\leq 30^\circ$ (acc, gyro, mag jointly), gain $\mathcal{N}(1, 0.1^2)$, $p = 0.8$
Final personalization; +FT	20 steps on ϕ_k; 20 steps on h
New users	
Few-shot labels; steps	1, 5, 10 windows per activity; 60 (30 for UniMTS)
Centralized references	
Epochs; early stopping	25 (20 / 15, UniMTS 8,000 windows per epoch), cosine; patience 5
Batch; learning rate	128 (UniMTS 64); scratch 10^{-3}, fine-tuning 3×10^{-4}
HALO pretraining (pool users)	
Schedule	40 epochs, batch 128, AdamW lr 3×10^{-4} (cosine)
Protocol A (published)	
Clients; rounds	all 120 users, 50 % per round; 100 rounds
MLP baselines	5 local epochs, batch 32 (as published)

TABLE X
HARBOX UNDER THE PUBLISHED PROTOCOL (PROTOCOL A: RANDOM 72/8/20 SPLIT OF 50–500 WINDOWS PER USER, ALL 120 USERS, 100 ROUNDS, HALF OF THE USERS PER ROUND). MEAN TEST ACCURACY OVER USERS AND OF THE WORST-SERVED 10 %. PUBLISHED NUMBERS ARE COPIED FROM FEDCHAR [28] (BENIGN SETTING); *ours* ARE RE-RUNS UNDER THE SAME PROTOCOL. SORTED BY ACCURACY.

Method	Model	Acc.	Worst-10 %	Source
FedAvg	MLP (300)	0.694	–	published [28]
FedAvg	MLP	0.815	0.343	ours
FedAvg	LIMU-BERT	0.819	0.341	ours
V0 Full FT	LIMU-BERT	0.831	0.374	ours
Local	MLP	0.878	0.631	ours
Centralized, scratch (not private)	MLP	0.881	0.564	ours
Centralized, scratch (not private)	LIMU-BERT	0.892	0.572	ours
Centralized, pretrained FT (not private)	LIMU-BERT	0.896	0.587	ours
FedCHAR (hierarchical clustering + Ditto)	MLP (300)	0.917	–	published [28]
Ditto	MLP	0.935	0.714	ours
V5 FedPEFT-IMU	LIMU-BERT	0.940	0.734	ours
V5 FedPEFT-IMU, LoRA on all layers	LIMU-BERT	0.941	0.751	ours
Ditto (pretrained)	LIMU-BERT	0.950	0.764	ours

augmentation, label-skew handling), so the comparison below is conservative for the baselines. Every personalized model we trained exceeds the best published result (FedCHAR, 0.917), without its hierarchical clustering. Ditto, the method on which FedCHAR builds, reaches 0.935 on the MLP. FedPEFT-IMU on LIMU-BERT reaches 0.940 (0.941 with all-layer LoRA). Ditto started from the same pretrained LIMU-BERT reaches 0.950. The same table exposes the weakness of the protocol: *purely local* training, with no federation at all, already reaches 0.878, and centralized training on the pooled data (0.881–0.896) stays below the personalized models. With 79.7 % of the test windows overlapping a training window, Protocol A largely measures how well a model memorizes a user's recording. A personal copy of every weight (Ditto) is the best memorizer, and its lead over FedPEFT-IMU (0.009) disappears under the leakage-free Protocol B (Section V-C). We therefore report Protocol A for comparability only and base all conclusions on Protocol B.

B. Leakage-Free Benchmark

Table XI reports the main grid under Protocol B.

FedPEFT-IMU is the best of the methods built on a pretrained encoder (V0–V5) on every encoder. It reaches a mean user accuracy of 0.861 on LIMU-BERT, 0.875 on ssl-wearables and 0.812 on HALO, which is 0.068, 0.065 and 0.098 above full federated fine-tuning (V0). This holds although V0 is scored after every client fine-tuned its classifier locally ("+FT"). On UniMTS, the slowest encoder, with only 40 rounds of 4 local steps, the margin shrinks to 0.001 accuracy and 0.004 macro-F1 over V3, but the worst-served 10 % of the clients still gain

TABLE XI

HARBox, leakage-free Protocol B (84 clients, per-activity chronological split with a one-window gap). Each cell: mean user accuracy / pooled macro-F1 on the clients' test parts. Methods with a shared classifier are scored after every client fine-tuned that classifier on its own training part ("+FT"), so every row compares personalized models. Best per encoder in bold. Budgets differ between encoders (Table IX), so compare methods within a column.

Method	LIMU-BERT	ssl-wearables	UniMTS	HALO	MLP
Local	– / –	– / –	– / –	– / –	0.824 / 0.824
FedAvg	0.779 / 0.803	0.783 / 0.805	0.725 / 0.759	0.573 / 0.577	0.766 / 0.793
Ditto	– / –	– / –	– / –	– / –	**0.868** / **0.875**
V0 Full FT	0.793 / 0.813	0.810 / 0.823	0.798 / 0.817	0.714 / 0.738	– / –
Ditto (pretrained)	**0.871** / **0.877**	– / –	– / –	– / –	– / –
V1 Head only	0.743 / 0.759	0.742 / 0.761	0.668 / 0.687	0.657 / 0.680	– / –
V2 LoRA naive	0.764 / 0.783	0.819 / 0.833	0.797 / 0.817	0.699 / 0.724	– / –
V3 LoRA exact	0.767 / 0.786	0.816 / 0.831	0.806 / 0.823	0.705 / 0.727	– / –
V4 Hetero ranks	0.752 / 0.770	0.809 / 0.825	0.788 / 0.809	0.677 / 0.703	– / –
V5 FedPEFT-IMU	0.861 / 0.863	**0.875** / **0.879**	**0.807** / **0.827**	**0.812** / **0.813**	– / –
V3, LoRA on all layers	0.796 / 0.817	– / –	– / –	– / –	– / –
V5 FedPEFT-IMU, LoRA on all layers	0.870 / 0.873	– / –	– / –	– / –	– / –
Centralized, scratch	0.833 / 0.847	0.832 / 0.840	0.795 / 0.812	0.681 / 0.699	0.800 / 0.819
Centralized, pretrained FT	0.814 / 0.829	0.847 / 0.857	0.824 / 0.836	0.813 / 0.824	–

TABLE XII

New users (24 held-out users, Protocol B): mean per-user macro-F1 with zero, 1, 5 and 10 labelled windows per activity. "transd.": the same V5 model with the transductive zero-shot rule (Eq. (20)), which also initializes the few-shot head.

Encoder	Model	0	1	5	10
LIMU-BERT	FedAvg	0.672	0.802	0.841	0.828
LIMU-BERT	V0	0.688	0.759	0.825	0.829
LIMU-BERT	V3	0.675	0.756	0.814	0.807
LIMU-BERT	V5	0.591	0.760	0.832	0.854
LIMU-BERT	V5, transd.	0.626	0.758	0.825	0.858
LIMU-BERT	*Central*	0.709	0.827	0.868	0.874
ssl-wearables	FedAvg	0.736	0.782	0.818	0.825
ssl-wearables	V0	0.797	0.852	0.857	0.872
ssl-wearables	V3	0.788	0.845	0.866	0.874
ssl-wearables	V5	0.775	0.796	0.843	0.860
ssl-wearables	V5, transd.	0.776	0.803	0.847	0.862
ssl-wearables	*Central*	0.784	0.845	0.857	0.856
UniMTS	FedAvg	0.706	0.806	0.826	0.818
UniMTS	V0	0.760	0.822	0.850	0.859
UniMTS	V3	0.762	0.798	0.833	0.845
UniMTS	V5	0.715	0.770	0.788	0.827
UniMTS	V5, transd.	0.732	0.789	0.814	0.845
UniMTS	*Central*	0.773	0.839	0.841	0.840
HALO	FedAvg	0.443	0.568	0.572	0.605
HALO	V0	0.568	0.712	0.778	0.768
HALO	V3	0.527	0.735	0.773	0.780
HALO	V5	0.534	0.764	0.817	0.815
HALO	V5, transd.	0.554	0.745	0.822	0.822
HALO	*Central*	0.556	0.668	0.732	0.760
MLP	FedAvg	0.669	0.775	0.824	0.836
MLP	*Central*	0.694	0.807	0.866	0.866

TABLE XIII

Component ablation of FedPEFT-IMU (LIMU-BERT, Protocol B, one component removed at a time). Acc.: mean user accuracy; Δ: macro-F1 change against the complete V5; new users: mean per-user macro-F1 with 0 and 10 labelled windows per activity.

Variant	Acc.	F1	Δ	W-10 %	0	10
V5 (complete)	0.861	0.863	–	0.533	0.591	0.854
+ transductive zero-shot (same model)	0.861	0.863	± 0	0.533	0.626	0.858
− prototype alignment ($\lambda = 0$)	0.859	0.862	-0.001	0.555	0.596	0.844
− local norm and input adapter	0.833	0.838	-0.025	0.461	0.602	0.859
− personal classifier (shared h)	0.862	0.866	+0.004	0.550	0.596	0.852
− device tiers (all $r = 8$)	0.863	0.866	+0.003	0.542	0.618	0.852
− training augmentation	0.872	0.873	+0.011	0.609	0.581	0.845
− averaged local parts for new users	0.861	0.863	-0.000	0.538	0.575	0.843
− augmentation in the last personal update	0.862	0.865	+0.002	0.580	0.591	0.853
+ average personal classifier for new users	0.865	0.866	+0.003	0.544	0.593	0.856
+ federated global classifier (dual head)	0.850	0.854	-0.009	0.507	0.598	0.843
+ LoRA on all linear layers (not only W_q, W_v)	0.870	0.873	+0.011	0.552	0.635	0.858
+ private LoRA copy in the last personal update	0.860	0.863	+0.000	0.517	0.589	0.857
+ private LoRA copy, 3× longer personal update	0.862	0.866	+0.003	0.539	0.592	0.854
V4 (− all local parts and prototypes)	0.752	0.770	-0.093	0.281	0.635	0.822
V0 full fine-tuning	0.793	0.813	-0.050	0.294	0.688	0.829
V0 − training augmentation	0.786	0.807	-0.056	0.294	0.685	0.833

(0.370 vs. 0.346 accuracy). On LIMU-BERT and ssl-wearables, V5 also exceeds the centralized models trained on the pooled data of all clients (0.833 and 0.847 at best), and on HALO it matches the pretrained centralized model (0.813). Pooling the data removes the communication constraint but not the heterogeneity between users. A model that is personal to every user addresses the heterogeneity.

Federated LoRA without personalization. With a shared classifier, federated LoRA (V2, V3) matches or beats full fine-tuning on the two convolutional encoders (macro-F1 0.833 vs. 0.823 on ssl-wearables, 0.823 vs. 0.817 on UniMTS) but not on the two Transformers (0.786 vs. 0.813 on LIMU-BERT, 0.727 vs. 0.738 on HALO). Naive factor averaging (V2) and exact aggregation (V3) are within 0.006 macro-F1 on every encoder: at these ranks and this number of local steps the cross terms of Eq. (7) are small. Heterogeneous ranks (V4) cost

TABLE XIV
CLIENT COST OF FULL FINE-TUNING (V0) AND FEDPEFT-IMU (V5) ON HARBOX: UPLINK PER CLIENT UPDATE, PEAK TRAINING MEMORY (MB) AND TIME PER LOCAL UPDATE (S).

	Uplink			Memory		Time	
Encoder	V0	V5	Ratio	V0	V5	V0	V5
LIMU-BERT	204 kB	26 kB	8×	70	63	0.43	0.42
ssl-wearables	42.1 MB	811 kB	52×	389	480	1.15	1.72
UniMTS	20.9 MB	555 kB	38×	1631	1074	2.24	3.36
HALO	18.5 MB	194 kB	95×	457	348	1.42	0.88

0.006–0.024 macro-F1 against homogeneous rank 8 (V3).

LoRA placement matters more than the aggregation rule. LIMU-BERT shares one Transformer block across its four layers, so adapting W_q and W_v gives only 2.3 k LoRA parameters. Wrapping all six linear layers of the block (8.1 k parameters) raises V3 from 0.786 to 0.817 macro-F1, above full fine-tuning (0.813), and V5 from 0.863 to 0.873. The uplink grows from 26 to 39 kB per update, still 5.2× less than full fine-tuning. Raising the rank instead helps less: on W_q, W_v, V3 gains steadily from rank 2 to 16 (0.769, 0.782, 0.786, 0.798 macro-F1), but rank 16 (4.6 k parameters) stays 0.019 below all-layer rank 8.

The strongest baseline. Ditto started from the pretrained LIMU-BERT keeps a personal copy of the whole model on every client. On split 0 it reaches 0.871 accuracy and 0.877 macro-F1, and FedPEFT-IMU with all-layer LoRA matches it (0.870, 0.873). Over three splits (Table XV), FedPEFT-IMU is level or ahead on every enrolled-client metric (0.877 vs. 0.875 accuracy, 0.883 vs. 0.882 macro-F1, 0.581 vs. 0.569 worst-10 %), with 5.2× less uplink and one local update per round instead of two. On the MLP, Ditto (0.868) is likewise far ahead of FedAvg (0.766) and of local training (0.824): under a leakage-free split, personalization helps, and so does federation.

C. Robustness Across User Splits

Table XV repeats the LIMU-BERT and MLP experiments on two further user splits (a new assignment of users to clients, held-out and pool users each time). The ranking is stable. FedPEFT-IMU reaches 0.868 ± 0.012 mean user accuracy, against 0.784 ± 0.013 for V0 and 0.769 ± 0.012 for FedAvg: its gain of 0.084 over V0 is about seven standard deviations of the split-to-split variation. With all-layer LoRA, FedPEFT-IMU reaches 0.877 ± 0.015, level with Ditto on the pretrained encoder (0.875 ± 0.008); with 10 labelled windows per activity, it is the best model for new users (0.852 vs. 0.827–0.845). The new-user scores vary much more between splits (±0.04–0.06 zero-shot), because each split has only 24 held-out users.

D. Fairness

Personalization matters most for the worst-served users. Averaged over the three splits, the mean accuracy of the 10 % worst clients is 0.275–0.328 for the shared LIMU-BERT models (FedAvg, V0, V3) and 0.284 for FedAvg on the MLP, against 0.568 for FedPEFT-IMU, 0.567 for Ditto and 0.551 for local training. The worst clients of the shared models score far below chance on their test parts (e.g., 0.09 for one user). This is consistent with self-labelling errors or unusual phone placement in crowd-sourced data, which a shared classifier cannot absorb but a personal model can.

E. New Users

Table XII reports the held-out users, who took part in no training.

Zero-shot, the shared classifier of full fine-tuning transfers better than FedPEFT-IMU's nearest-prototype rule on every encoder: 0.688 vs. 0.591 on LIMU-BERT, 0.797 vs. 0.775 on ssl-wearables, 0.760 vs. 0.715 on UniMTS and 0.568 vs. 0.534 on HALO. The embedding of FedPEFT-IMU is trained jointly with personal classifiers and local norms, which a new user does not have yet. Three measures narrow the gap without labels. Initializing a new user with the clients' average local parameters (Section III-H) raises zero-shot from 0.575 to 0.591. The transductive rule (Eq. (20)) raises it further to 0.626 (0.699 ± 0.063 over the three splits, against 0.743 ± 0.048 for V0), UniMTS's from 0.715 to 0.732 and HALO's from 0.534 to 0.554. On ssl-wearables, where AdaBN has already adapted every normalization layer to the user, it adds almost nothing (0.775 to 0.776). All-layer LoRA, together with the transductive rule, reaches 0.665 on LIMU-BERT (0.727 ± 0.054 over three splits), within 0.02 of V0 (0.743) and of Ditto on the pretrained encoder (0.745).

With labels, the picture changes. With 10 labelled windows per activity (20 s), FedPEFT-IMU reaches 0.854 on LIMU-BERT, against 0.829 for V0 and 0.807 for V3. On HALO it is the best model at every labelled budget (0.764, 0.817 and 0.815 with 1, 5 and 10 windows). On the two BatchNorm encoders, the shared models remain ahead with labels (0.872–0.874 vs. 0.860 on ssl-wearables, 0.859 vs. 0.827 on UniMTS, whose few-shot budget is cut to 30 steps). Keeping the AdaBN statistics fixed during few-shot adaptation (Section III-H) matters here: when a batch of a few labelled windows was allowed to overwrite them, the 1-shot score of ssl-wearables fell below its zero-shot score (0.751); with the fix, it is 0.796.

F. Communication, Memory and Time

Table XIV compares the client cost of FedPEFT-IMU with full fine-tuning. The uplink per update shrinks 8× on LIMU-BERT (204 to 26 kB), 52× on ssl-wearables (42.1 MB to 811 kB), 38× on UniMTS (20.9 MB to 555 kB) and 95× on HALO (18.5 MB to 194 kB). Peak training memory falls by 34 % on UniMTS and 24 % on HALO, where the optimizer state of the full model matters, and by 10 % on LIMU-BERT. On ssl-wearables, the V5 run peaked higher (480 vs. 389 MB). Memory on the convolutional encoders includes cuDNN workspaces, which vary between runs, and activations dominate for every encoder, because LoRA still back-propagates through the whole network. The time per local update does not separate the methods reliably: the runs shared one GPU with other experiments, and the same configuration varied by up to ±50 % between runs. Over a round, compute time is small next to the transfer of a full model on a mobile link.

TABLE XV
REPEATED USER SPLITS (PROTOCOL B, SEEDS 0–2; n: NUMBER OF SPLITS): MEAN $\pm$ STANDARD DEVIATION. ENROLLED CLIENTS: MEAN USER ACCURACY, MACRO-F1 AND THE MEAN ACCURACY OF THE WORST-SERVED 10 % (SHARED CLASSIFIERS AFTER "+FT"). NEW USERS: MEAN PER-USER MACRO-F1 ZERO-SHOT (V5: TRANSDUCTIVE RULE) AND WITH 10 LABELLED WINDOWS PER ACTIVITY.

			Enrolled clients			New users	
Method	Model	n	Acc.	Macro-F1	Worst-10 %	0-shot	10-shot
Local	MLP	3	0.828 ± 0.012	0.834 ± 0.013	0.551 ± 0.029	–	–
FedAvg	MLP	3	0.745 ± 0.018	0.776 ± 0.015	0.284 ± 0.007	0.720 ± 0.045	0.832 ± 0.013
Ditto	MLP	3	0.868 ± 0.005	0.877 ± 0.003	0.567 ± 0.018	0.728 ± 0.041	0.837 ± 0.006
FedAvg	LIMU-BERT	3	0.769 ± 0.012	0.796 ± 0.006	0.275 ± 0.011	0.729 ± 0.051	0.840 ± 0.013
V0 Full FT	LIMU-BERT	3	0.784 ± 0.013	0.808 ± 0.006	0.328 ± 0.040	0.743 ± 0.048	0.840 ± 0.014
Ditto (pretrained)	LIMU-BERT	3	0.875 ± 0.008	0.882 ± 0.006	0.569 ± 0.038	0.745 ± 0.051	0.834 ± 0.013
V3 LoRA exact	LIMU-BERT	3	0.759 ± 0.007	0.784 ± 0.005	0.304 ± 0.008	0.727 ± 0.046	0.827 ± 0.020
V5 FedPEFT-IMU	LIMU-BERT	3	0.868 ± 0.012	0.871 ± 0.010	0.568 ± 0.052	0.699 ± 0.063	0.845 ± 0.025
V5 FedPEFT-IMU, LoRA on all layers	LIMU-BERT	3	0.877 ± 0.015	0.883 ± 0.014	0.581 ± 0.053	0.727 ± 0.054	0.852 ± 0.020

G. Ablation

Table XIII removes or adds one component of FedPEFT-IMU at a time on LIMU-BERT, with LoRA on W_q, W_v as in the main grid.

Local normalization and input adapter are the most important component. Without them, macro-F1 drops by 0.025 and the worst-$10\,\%$ accuracy from 0.533 to 0.461. Without all local parts and prototypes (V4), macro-F1 drops by 0.093.

The personal classifier adds little once the norms are local. A shared classifier that every client fine-tunes before testing reaches the same accuracy (0.862 vs. 0.861). *Prototype alignment* does not change the enrolled clients (-0.001) or zero-shot ($+0.005$), but it raises the 10-shot score by 0.010. *Device tiers* cost nothing: a single rank 8 for every device gives the same macro-F1 ($+0.003$), so the low-rank devices of FedPEFT-IMU are not penalized.

Augmentation is a trade-off. Without it, the enrolled clients gain 0.011 macro-F1, but new users lose 0.010 zero-shot and 0.009 with 10 labels. We keep it, because the paper targets new users as well as enrolled ones. *Averaged local parts for new users* raise every new-user score ($+0.016$ zero-shot, $+0.011$ at 10 shots) for a one-time upload of $2\,\mathrm{kB}$ per client.

Negative results. Three further ideas did not help. An average of the clients' personal classifiers as the zero-shot rule changed nothing ($+0.002$). A second, federated classifier for new users (dual head) cost 0.009 macro-F1 for a $+0.007$ zero-shot gain. A private copy of the LoRA adapters, fine-tuned in the last personal update (the PEFT counterpart of Ditto's personal model), did not improve the enrolled clients either with 20 or with 60 steps ($\leq +0.003$). By contrast, all-layer LoRA ($+0.011$ macro-F1, $+0.044$ zero-shot) and the transductive zero-shot rule ($+0.035$) are clear gains.

H. Discussion and Limitations

Evaluation protocol. The largest effect in this study is not a method but the protocol: under the random split of the published results, purely local training approaches the best published federated method, while under a leakage-free split every model loses several points and federation clearly beats local training. We recommend that future HARBox results report the per-activity chronological split (or another split that keeps overlapping windows on one side) alongside any random split.

New users versus enrolled users. Local normalization and personal classifiers are what make FedPEFT-IMU strong for enrolled users, and they are also why its zero-shot accuracy for brand-new users is lower than that of a shared model. Averaged local parts and the transductive rule narrow this gap, and a few labelled windows close it on LIMU-BERT and HALO. Deployments that must serve many unlabelled newcomers may combine FedPEFT-IMU with a shared fallback model.

Personalization depth. Ditto personalizes every weight and pays with a full-model uplink and two local updates per round. FedPEFT-IMU personalizes only the norms, the input adapter and the classifier, on top of shared low-rank adapters, and reaches the same accuracy once the adapters cover the whole Transformer block. A private copy of the adapters adds nothing, which suggests that on HARBox the user-specific part of the signal lives in the input statistics and the decision rule rather than in the encoder weights.

Downlink. Exact aggregation changes the frozen weights, and our accounting conservatively charges their full dense size at every download. For the convolutional encoders, this makes the downlink comparable to full fine-tuning. A factored encoding of the residual (Section III-K) is left for future work.

Scope. The large encoders train with reduced budgets on our single $6\,\mathrm{GB}$ GPU, so absolute numbers across encoders are not budget-matched. HALO is our re-implementation, pretrained only on the unlabelled windows of 12 pool users. LIMU-BERT and the MLP baselines are evaluated on three user splits; the other encoders on one.

VI. CONCLUSION

[Conclusion: to be written.]

REFERENCES

[1] X. Li, M. Jiang, X. Zhang, M. Kamp, and Q. Dou, "FedBN: Federated learning on non-IID features via local batch normalization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.

[2] L. Collins, H. Hassani, A. Mokhtari, and S. Shakkottai, "Exploiting shared representations for personalized federated learning," in *Proc. 38th Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 139, 2021, pp. 2089–2099.

[3] M. G. Arivazhagan, V. Aggarwal, A. K. Singh, and S. Choudhary, "Federated learning with personalization layers," arXiv:1912.00818, 2019.

[4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

[5] Y. Tan, G. Long, L. Liu, T. Zhou, Q. Lu, J. Jiang, and C. Zhang, "FedProto: Federated prototype learning across heterogeneous clients," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 36, no. 8, 2022, pp. 8432–8440.

[6] R. Singhal, K. Ponkshe, and P. Vepakomma, "FedEx-LoRA: Exact aggregation for federated and efficient fine-tuning of large language models," in *Proc. 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

[7] N. Halko, P.-G. Martinsson, and J. A. Tropp, "Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions," *SIAM Review*, vol. 53, no. 2, pp. 217–288, 2011.

[8] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, ser. PMLR, vol. 54, 2017, pp. 1273–1282.

[9] J. Bai, D. Chen, B. Qian, L. Yao, and Y. Li, "Federated fine-tuning of large language models under heterogeneous tasks and client resources," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[10] C. Eckart and G. Young, "The approximation of one matrix by another of lower rank," *Psychometrika*, vol. 1, no. 3, pp. 211–218, 1936.

[11] S. Chan, H. Yuan, C. Tong, A. Acquah, A. Schonfeldt, J. Gershuny, and A. Doherty, "CAPTURE-24: A large dataset of wrist-worn activity tracker data collected in the wild for human activity recognition," *Scientific Data*, vol. 11, p. 1135, 2024.

[12] A. K. Menon, S. Jayasumana, A. S. Rawat, H. Jain, A. Veit, and S. Kumar, "Long-tail learning via logit adjustment," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.

[13] J. Ren, C. Yu, S. Sheng, X. Ma, H. Zhao, S. Yi, and H. Li, "Balanced meta-softmax for long-tailed visual recognition," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 4175–4186.

[14] J. Zhang, Z. Li, B. Li, J. Xu, S. Wu, S. Ding, and C. Wu, "Federated learning with label distribution skew via logits calibration," in *Proc. 39th Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 162, 2022, pp. 26 311–26 329.

[15] X.-C. Li and D.-C. Zhan, "FedRS: Federated learning with restricted softmax for label distribution non-IID data," in *Proc. 27th ACM SIGKDD Conf. Knowledge Discovery & Data Mining*, 2021, pp. 995–1005.

[16] X. Shang, Y. Lu, G. Huang, and H. Wang, "Federated learning on heterogeneous and long-tailed data via classifier re-training with federated features," in *Proc. 31st Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2022, pp. 2218–2224.

[17] R. Walmsley, S. Chan, K. Smith-Byrne, R. Ramakrishnan, M. Woodward, K. Rahimi, T. Dwyer, D. Bennett, and A. Doherty, "Reallocation of time between device-measured movement behaviours and risk of incident cardiovascular disease," *British Journal of Sports Medicine*, vol. 56, no. 18, pp. 1008–1017, 2022.

[18] A. J. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," *IEEE Transactions on Information Theory*, vol. 13, no. 2, pp. 260–269, 1967.

[19] Y. Li, N. Wang, J. Shi, X. Hou, and J. Liu, "Adaptive batch normalization for practical domain adaptation," *Pattern Recognition*, vol. 80, pp. 109–117, 2018.

[20] Y. Iwasawa and Y. Matsuo, "Test-time classifier adjustment module for model-agnostic domain generalization," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 2427–2440.

[21] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.

[22] H. Xu, P. Zhou, R. Tan, M. Li, and G. Shen, "LIMU-BERT: Unleashing the potential of unlabeled data for IMU sensing applications," in *Proc. 19th ACM Conf. Embedded Networked Sensor Systems (SenSys)*, 2021, pp. 220–233.

[23] H. Yuan, S. Chan, A. P. Creagh, C. Tong, A. Acquah, D. A. Clifton, and A. Doherty, "Self-supervised learning for human activity recognition using 700,000 person-days of wearable data," *npj Digital Medicine*, vol. 7, p. 91, 2024.

[24] X. Zhang, D. Teng, R. R. Chowdhury, S. Li, D. Hong, R. K. Gupta, and J. Shang, "UniMTS: Unified pre-training for motion time series," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[25] Z. Ding, L. Zhang, and X. Ouyang, "HALO: A heterogeneity-aware language-aligned IMU foundation model for open-set human activity recognition," arXiv:2608.27233, 2026.

[26] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2017, pp. 1175–1191.

[27] X. Ouyang, Z. Xie, J. Zhou, J. Huang, and G. Xing, "ClusterFL: A similarity-aware federated learning system for human activity recognition," in *Proc. 19th Annual Int. Conf. Mobile Systems, Applications, and Services (MobiSys)*, 2021, pp. 54–66.

[28] Y. Li, X. Wang, and L. An, "Hierarchical clustering-based personalized federated learning for robust and fair human activity recognition," *Proc. ACM Interactive, Mobile, Wearable and Ubiquitous Technologies (IMWUT)*, vol. 7, no. 1, pp. 20:1–20:38, 2023.

[29] A. Stisen, H. Blunck, S. Bhattacharya, T. S. Prentow, M. B. Kjærgaard, A. Dey, T. Sonne, and M. M. Jensen, "Smart devices are different: Assessing and mitigating mobile sensing heterogeneities for activity recognition," in *Proc. 13th ACM Conf. Embedded Networked Sensor Systems (SenSys)*, 2015, pp. 127–140.

[30] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 16 000–16 009.

[31] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," arXiv:1807.03748, 2018.

[32] S. Yan, Y. Xiong, and D. Lin, "Spatial temporal graph convolutional networks for skeleton-based action recognition," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 32, no. 1, 2018.

[33] C. Guo, S. Zou, X. Zuo, S. Wang, W. Ji, X. Li, and L. Cheng, "Generating diverse and natural 3D human motions from text," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 5152–5161.

[34] T. Li, S. Hu, A. Beirami, and V. Smith, "Ditto: Fair and robust federated learning through personalization," in *Proc. 38th Int. Conf. Machine Learning (ICML)*, ser. PMLR, vol. 139, 2021, pp. 6357–6368.

[35] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, K. H. Li, T. Parcollet, P. P. B. de Gusmão, and N. D. Lane, "Flower: A friendly federated learning research framework," arXiv:2007.14390, 2020.